Project Overview

This project is a full-stack e-commerce web application where users can browse books, add them to a cart, place orders, and leave reviews.

The project is divided into clear backend and frontend responsibilities and follows RESTful API best practices.

Backend (Node.js + Express.js + MongoDB)

1- Authentication & Authorization

Features:

- User registration
- User login
- User logout
- JWT-based authentication
- Role-Based Access Control (Admin / User)
- Protect private routes using authentication middleware
- Admin-only routes for managing books, authors and categories

---

2- Database Design

- Students are responsible for designing the database schemas.
- Only functionality and behavior are defined in this document.
- Schema design decisions must be justified during project presentation.
- ERD is required
-

3- RESTful API Design

Design the following API modules:

User Routes

- Register

  Registration form should include email(unique), firstName, lastName, dob, password
  Bonus: Add email verification before login

- Login
- Update profile

Category Routes

- Create category (Admin only) - name should be unique
- Get all categories
- Update category (Admin only)
- Delete category (Admin only) - handle books linked to this category properly

Author Routes

- Create author (Admin only) should have name and bio
- Update Author (Admin only)
- List authors

Book Routes

- Create book (Admin only)

  Book creation form should include name, author, category, and book cover

- Get all books

  Filter by categories, authors and price range, Search by name

- Get single book
- Update book (Admin only)
- Delete book (Admin only)

Cart Routes

- Add book to cart
- Remove book from cart
- View cart

Order Routes

- Place order

  Order should be placed with shipping details and initial status (processing) and initial payment status (pending)
  Payment method is selected by default as: (COD) bonus for integrating with a payment gateway => initial payment status would be (success)

- View orders history
- Get all orders (Admin only)
- Update order status processing -> out for delivery -> delivered, also payment status pending -> success (Admin only)

Review Routes

- Add review

  Only users who purchased the book can add a review and rating (stars and optional comment)

- View reviews
- Delete review

3- Middleware

Create custom middleware for:

- JWT Authentication
- Authorization
- Centralized Error Handling
- Logging (using `pino` or `winston`)

4- Validation

Use:

- Mongoose validations
- Schema validation library (Joi / Ajv)

Validate:

- Email format
- Password strength
- Book price (positive number)
- Stock (integer)
- Review rating (1–5)
- Review length (max 500 characters)

5- File Handling

- Upload book covers using `cloudinary (pre-signed url)`

6- Advanced Core Features

- Pagination for books
- Filtering books
- Use MongoDB transactions when placing orders:
    - Reduce stock
    - Create order
    - Commit transaction

Frontend (Angular)

1- User Interface Pages

Home Page

- List popular books
- List popular authors

Books Page

- List books
- Search (By Name)
- Filter (multiple Categories - authors - price range)

Book Details Page

- Book information

  Name, cover, author details, category rating and reviews

- Add to cart
- Display reviews and add reviews

Cart Page

- View cart items
- Proceed to Checkout button

Checkout Page

- View cart items
- View payment summary
- Add shipping address

Order History Page

- View previous orders

2- Admin Panel

Admin Panel Should include:

- CRUD for books
- CRUD for authors
- CRUD for categories
- Order management: Listing users Orders and updating order status

---

3- Authentication

- Login form
- Registration form
- JWT handling via Angular services
- Store token securely

4- API Integration

- Use Angular `HttpClient`
- Handle errors properly
- Display success/error messages

5- Review System

- Review form (only if user purchased book)
- Display reviews
- Delete own review

6- UI Enhancements

- Lazy loading modules
- Use Angular Material or Bootstrap
- Responsive design

General Rules

- Pagination must be server-side
- Think about indexing (email, book title, etc.)
- Use proper error handling
- Backend and frontend are maintained in separate GitHub repositories.
- Clean Git commit history (make sure every commit has a clear message, everyone should contribute to the project)
- Lint your code
- Deploy backend & frontend
- Write proper README, .env.example
- Create Postman/Bruno collection or use swagger